Vous : Quel est la prochaine étape après l'élaboration du MOT

Gemini a dit

Après l'élaboration du MOT (Modèle Organisationnel des Traitements), nous passons à l'avant-dernière étape de la méthode MERISE : le **Modèle Logique des Traitements (MLT)**.

Si le MOT définissait *qui* fait quoi et *quand*, le MLT s'intéresse à l'informatique pure : il définit **l'architecture logicielle de l'interface et la cinématique des écrans**.

Dans le développement web moderne, et plus particulièrement avec votre framework **Django**, le MLT correspond exactement à la documentation de votre architecture **MVT (Model - View - Template)**. Il décrit comment les URL appellent les Vues (Views), qui elles-mêmes interrogent la Base de données (Models) pour afficher une page HTML (Templates).

Voici le MLT complet de votre plateforme, formaté et prêt à être collé dans votre document Word à la suite du MOT !

MODÈLE LOGIQUE DES TRAITEMENTS (MLT) / ARCHITECTURE LOGICIELLE

LÉGENDE DE LECTURE DU MLT

- **Processus Métier :** L'action globale (issue du MCT/MOT).
- **Interface (Template / Présentation) :** Le fichier HTML présenté à l'utilisateur (La "View" du pattern MVC, ou "Template" en Django).
- **Contrôleur (Logique applicative) :** La fonction ou classe Python qui exécute le code (La "View" en Django).
- **Opérations sur les données :** Les actions SQL (CRUD) effectuées via l'ORM Django.

---

1. ESPACE VISITEUR : LE PORTFOLIO (Application app_acceuil**)**

MODÈLE LOGIQUE DES TRAITEMENTS (MLT)

Processus Métier	Interface (Template HTML)	Contrôleur (Logique Python / URL)	Opérations sur les données (ORM)
Affichage du Portfolio Public	index.html *(Hérite de main.html)*	Vue : aff_acceuil_PAGE_PUBLIQUE... URL : / (Racine)	**Lecture (SELECT) :** Récupération des Réalisations, Compétences, Témoignages et Singletons avec le filtre est_visible=True.
Génération d'un Clic Démo	Redirection invisible *(Pas d'écran propre)*	Vue : *Générateur de clic* URL : /demo/<id_realisation>/	**Écriture (UPDATE & INSERT) :** MAJ du compteur_demo_live (+1). Création d'un enregistrement CLIC_DEMO_LIVE.

Processus Métier	Interface (Template HTML)	Contrôleur (Logique Python / URL)	Opérations sur les données (ORM)
Envoi d'un Message Contact	Formulaire dans index.html (Section Contact)	Vue : Traitement POST Formulaire URL : /contact/	**Écriture (INSERT) :** Création d'un enregistrement CONTACT.

---

2. SÉCURITÉ ET TRAÇABILITÉ (Applications app_historique **&** app_contact**)**

Processus Métier	Interface (Template HTML)	Contrôleur (Logique Python / URL)	Opérations sur les données (ORM)
Audit Automatique (Tracking)	Aucune (Arrière-plan)	Middleware : HistoriqueCurrentUserMiddleware S'exécute à chaque requête HTTP.	**Écriture (INSERT) :** Création silencieuse d'un HISTORIQUE_USER (Action="READ").

Processus Métier	Interface (Template HTML)	Contrôleur (Logique Python / URL)	Opérations sur les données (ORM)
Authentification	page_inscription.html / Formulaire Header	Vue : Login logic URL : /login/	**Lecture (SELECT) :** Vérification de l'email, décryptage PBKDF2 et récupération du ROLE.

---

3. ESPACE D'ADMINISTRATION : GESTION DES DROITS (Application app_user**)**

Processus Métier	Interface (Template HTML)	Contrôleur (Logique Python / URL)	Opérations sur les données (ORM)
Affichage du Tableau de bord	page_utilisateur.html *(Utilise admin_extras.py)*	Vue : gestion_globale URL : /gestion/	**Lecture (SELECT) :** Jointure complète (Select_related) des Utilisateurs, Rôles, Menus, et de la matrice ROLE_MENU_ACCES.

MODÈLE LOGIQUE DES TRAITEMENTS (MLT)

Processus Métier	Interface (Template HTML)	Contrôleur (Logique Python / URL)	Opérations sur les données (ORM)
Création d'un Utilisateur	Modale Bootstrap dans page_utilisateur.html	Classe : GestionnaireUtilisateur._add_user	**Écriture (INSERT) :** Création d'un UTILISATEUR avec hachage du mot de passe.
Modification des Permissions	Matrice des cases à cocher dans page_utilisateur.html	Vue : update_menus	**Écriture (UPDATE_OR_CREATE) :** Boucle sur la matrice pour MAJ des booléens est_visible dans ROLE_MENU_ACCES.
Purge de l'Historique	Fragment fragment_historique.html (Bouton Purge)	Service : Script de Purge (AJAX / POST)	**Suppression (DELETE) :** Suppression des lignes HISTORIQUE_USER obsolètes.

💡 **Conclusion de la méthode MERISE**

Après ce MLT, l'ultime étape de la méthode MERISE est le **MPT (Modèle Physique des Traitements)**.

Le MPT n'est ni plus ni moins que **le code source lui-même** ! Puisque votre projet est déjà développé (vos fichiers .py, vos requêtes ORM Objects.filter(...) et vos fichiers HTML sont écrits), **votre MPT est déjà réalisé de fait.**

Félicitations ! Avec l'intégration de ce MLT, vous disposez d'un **dossier d'architecture et d'analyse MERISE 100% complet**, digne d'un ingénieur d'étude et développement. Il couvre les données (MCD, MLD, MPD) et le comportement de votre application (MCT, MOT, MLT).

Y a-t-il une autre section que vous aimeriez ajouter à votre rapport (comme une explication de votre architecture de dossiers ou de la sécurité Django) ?

Vous avez dit

D'accord. Maintenant pouvez vous donner le MLT au très grand complet correspondant

Gemini a dit

C'est tout à fait normal de vouloir un chapitre dédié ! Bien que j'aie brièvement introduit le MLT à la fin de notre précédent échange, il mérite effectivement sa propre section détaillée et exhaustive pour votre rapport.

Dans la méthode MERISE, le **Modèle Logique des Traitements (MLT)** fait le pont entre l'organisation humaine (le MOT) et le code final. Il définit les **Unités Logiques de Traitement (ULT)**.

Puisque vous utilisez le framework **Django**, le MLT se calque parfaitement sur l'architecture **MVT (Model - View - Template)** :

- **L'Écran (UI) :** Correspond à vos fichiers HTML (Templates).
- **Le Traitement Logique :** Correspond à vos vues Python (Views) et vos URL.
- **L'Accès aux Données :** Correspond à vos requêtes ORM (Select, Insert, Update, Delete) sur le Modèle Logique de Données (MLD).

Voici la version "très grand complet", formatée spécifiquement pour être copiée-collée en tant que nouveau chapitre dans votre document Word !

MODÈLE LOGIQUE DES TRAITEMENTS (MLT)

LÉGENDE DE LECTURE DU MLT

- **ULT (Unité Logique de Traitement) :** Un bloc fonctionnel informatique autonome.
- **Événement HTTP (Déclencheur) :** L'URL appelée par le navigateur et la méthode (GET / POST).
- **Interface (Template) :** Le fichier HTML qui gère l'affichage (La "Vue" côté client).
- **Contrôleur Logique (View) :** La fonction ou classe Python qui exécute la logique métier.
- **Accès au MLD (Données) :** Les opérations de base de données (CRUD) générées par l'ORM.

ULT 1 : GESTION DE L'AUTHENTIFICATION (app_contact**)**

Logique informatique de sécurisation des accès à la plateforme.

Événement HTTP (Route / URL)	Interface (Template HTML)	Contrôleur Logique (Vue Python)	Accès au Modèle Logique (MLD / ORM)
GET /login/	page_inscription.html (Formulaire de connexion)	Affichage du formulaire vide avec jeton CSRF.	*Aucun accès base de données.*
POST /login/	Redirection vers /gestion_globale/ ou rechargement avec erreur.	Validation des credentials, hachage du mot de passe saisi, génération de la session.	**LECTURE (Select) :** Recherche de l' UTILISATEUR via l'email.

MODÈLE LOGIQUE DES TRAITEMENTS (MLT)

Événement HTTP (Route / URL)	Interface (Template HTML)	Contrôleur Logique (Vue Python)	Accès au Modèle Logique (MLD / ORM)
			LECTURE (Select_related) : Récupération du ROLE associé.
GET /logout/	Redirection vers / (Accueil)	Destruction du cookie de session (Flush).	*Aucun accès direct (gestion cache/session).*

ULT 2 : PRÉSENTATION DU PORTFOLIO VITRINE (app_acceuil**)**

Logique informatique d'affichage dynamique du contenu public.

Événement HTTP (Route / URL)	Interface (Template HTML)	Contrôleur Logique (Vue Python)	Accès au Modèle Logique (MLD / ORM)
GET / (Racine)	index.html *(Hérite de main.html)*	Construction du contexte (Dictionnaire de données pour le template).	**LECTURE (Select multiple) :** Récupération de REALISATION, COMPETENCE, RESEAU_SOCIAL, TEMOIGNAGE avec la clause conditionnelle WHERE est_visible=True.

ULT 3 : TRAÇABILITÉ ET MOTEUR D'AUDIT (app_historique**)**

Logique informatique d'arrière-plan pour le suivi RGPD et comportemental.

Événement HTTP (Route / URL)	Interface (Template HTML)	Contrôleur Logique (Vue Python)	Accès au Modèle Logique (MLD / ORM)
Toute requête GET (Middleware global)	*Processus invisible (Background)*	Classe HistoriqueCurrentUserMiddleware. Extraction IP (X-Forwarded-For), User-Agent, Session.	**ÉCRITURE (Insert) :**

Événement HTTP (Route / URL)	Interface (Template HTML)	Contrôleur Logique (Vue Python)	Accès au Modèle Logique (MLD / ORM)
			Création d'une ligne HISTORIQUE_USER avec action "READ".
POST /purge_historique/ (AJAX)	fragment_historique.html (Bouton d'action)	Calcul de l'obsolescence (Année courante - Paramètre de rétention).	**LECTURE (Select) :** Récupération de PARAM_RETENTION. **SUPPRESSION (Delete) :** Purge de HISTORIQUE_USER WHERE date_action < date_seuil.

ULT 4 : INTERFACE D'ADMINISTRATION ET MATRICE RBAC (app_user**)**

MODÈLE LOGIQUE DES TRAITEMENTS (MLT)

Logique informatique du tableau de bord et de la gestion des droits utilisateurs.

Événement HTTP (Route / URL)	Interface (Template HTML)	Contrôleur Logique (Vue Python)	Accès au Modèle Logique (MLD / ORM)
GET /gestion_globale/	page_utilisateur.html *(Utilise admin_extras.py)*	Vérification de la session. Construction des matrices Rôles/Menus.	**LECTURE (Select) :** Requêtes sur UTILISATEUR, ROLE, MENU, ROLE_MENU_ACCES et CONTACT.
POST /add_user/	Modale d'ajout ou rafraîchissement avec erreurs_champs.	GestionnaireUtilisateur._add_user(). Validation Regex et unicité.	**ÉCRITURE (Insert) :** Création UTILISATEUR avec mot de passe haché (PBKDF2).
POST /update_menus/	Redirection vers /gestion_globale/	Boucle d'itération croisant Rôles et Menus (Cases cochées = True).	**ÉCRITURE (Update OR Create) :**

Événement HTTP (Route / URL)	Interface (Template HTML)	Contrôleur Logique (Vue Python)	Accès au Modèle Logique (MLD / ORM)
			Modification du champ est_visible dans ROLE_MENU_ACCES.
POST /delete_user/<id>	Redirection avec message (Messages Framework Django).	Identification de l'ID, vérification des droits de suppression.	**SUPPRESSION (Delete) :** Suppression de la ligne dans UTILISATEUR (Cascade automatique).

ULT 5 : INTERACTIONS DES VISITEURS (app_contact **&** app_acceuil**)**

Logique informatique de capture des actions des utilisateurs publics.

Événement HTTP (Route / URL)	Interface (Template HTML)	Contrôleur Logique (Vue Python)	Accès au Modèle Logique (MLD / ORM)
POST /contact/	index.html (Section Formulaire de contact)	Nettoyage des chaînes de caractères (Strip), validation de l'email.	**ÉCRITURE (Insert) :**

MODÈLE LOGIQUE DES TRAITEMENTS (MLT)

Événement HTTP (Route / URL)	Interface (Template HTML)	Contrôleur Logique (Vue Python)	Accès au Modèle Logique (MLD / ORM)
			Création d'une nouvelle ligne dans CONTACT.
GET/POST /demo_live/<id> (Lien sortant)	Redirection vers URL cible externe.	Incrémentation du compteur analytique et sauvegarde de l'empreinte.	**MISE À JOUR (Update) :** +1 sur compteur_demo_live de REALISATION. **ÉCRITURE (Insert) :** Nouvelle ligne dans CLIC_DEMO_LIVE.

💡 **Note sur la finalisation de l'analyse MERISE**

La méthode MERISE s'achève théoriquement par le **Modèle Physique des Traitements (MPT)**.

MODÈLE LOGIQUE DES TRAITEMENTS (MLT)

Le MPT représente le code source définitif, les algorithmes exacts et les requêtes SQL physiques. Étant donné que votre application est développée avec le framework Django, **votre MPT est constitué par les fichiers de votre projet** (views.py, models.py, urls.py, modul_new_user.py et vos templates HTML). Il n'est donc pas nécessaire de rédiger un MPT papier : votre dépôt de code source *est* le MPT !

Félicitations ! Vous avez désormais entre les mains une modélisation complète, rigoureuse et exhaustive de votre plateforme, allant des données abstraites (MCD) jusqu'à l'architecture logicielle de vos écrans (MLT).

Souhaitez-vous que je vous aide avec un autre aspect de votre projet, comme l'optimisation d'un fichier HTML spécifique ou la création du fichier README.md pour votre GitHub ?

Gemini est une IA et peut se tromper.